ASTRA polytechnic
member of ASTRA

Kampus Merdeka
INDONESIA JAYA
satu Indonesia
Semangat Astra Terpadu Untuk Indonesia

Modular Programing (*Procedure & Function*)

Prepared by : Radix R

Contents

- Introduction to Modular Programming
- Procedure
- Function

Intro

- “To achieve a larger goal, then the goal must be divided into smaller goals until the small goals are goals that can be achieved based on the conditions and potential that are available at that time.”

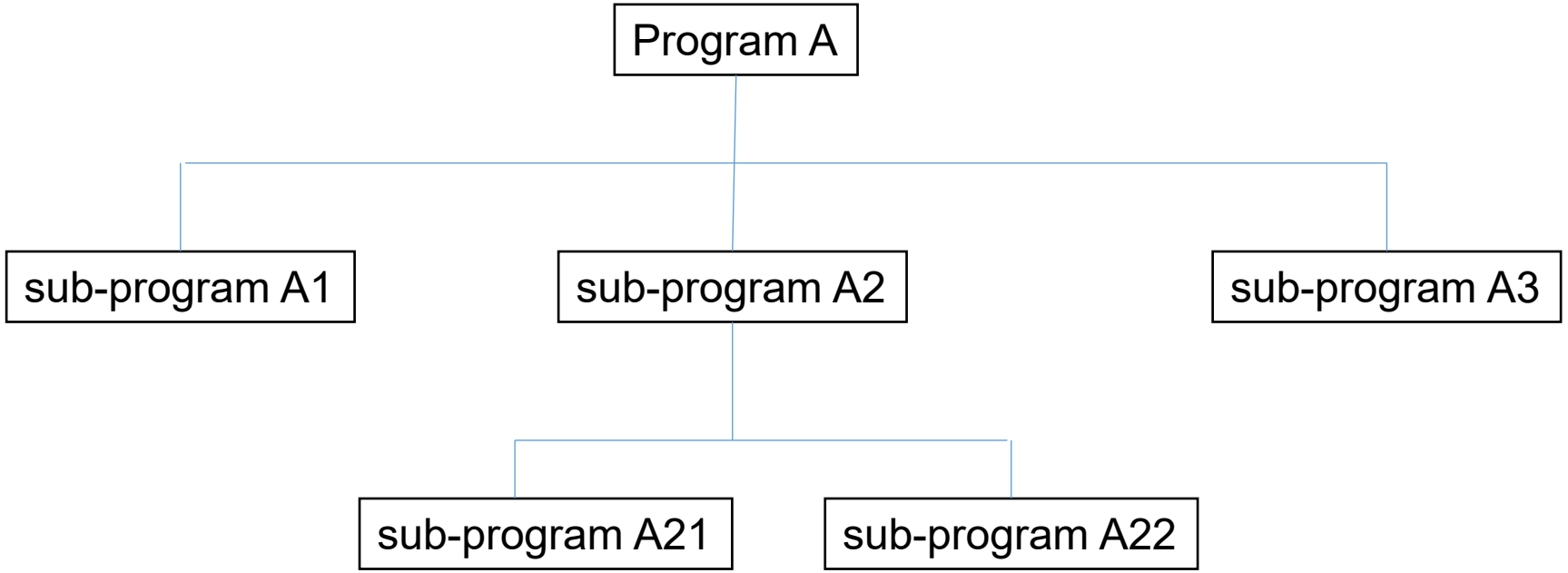
(Al-Khuwarizmi)

intro

- Large programs are difficult to handle, to understand, and to track mistakes.
- Programs should be broken down into several sub-programs.

- The technique for breaking a program into several programs is called modular programming.

Program Decomposition



Program Decomposition example

```
1   PROGRAM Switching
2   (Switch the value of A & B )
3
4   DECLARATION:
5       A, B, C : integer
6
7   ALGORITHM:
8       { suppose that A is filled with 8 and B is filled with 5 }
9       A ← 8
10      B ← 5
11
12      { print A and B values before switching }
13      write("A and B values before switching: ")
14      write("A = ", A)
15      write("B = ", B)
16
17      { Switch the value of A & B }
18      C ← A
19      A ← B
20      B ← C
21
22      { print A and B values after switching }
23      write("A and B values after switching: ")
24      write("A = ", A)
25      write("B = ", B)
```

Program Decomposition example

```
Procedure Filling(A, B: integer)
( Filling A & B with direct assignment )

DECLARATION:
    { none }

ALGORITHM:

    { suppose that A is filled with 8 and B is filled with 5 }
    A ← 8
    B ← 5
```

```
Procedure Printing(A, B: integer)
( mencetak nilai A & B )

DECLARATION:
    { none }

ALGORITHM:

    write("A = ", A)
    write("B = ", B)
```

```
Procedure Switching(A, B: integer)
(Switching the value of A & B)

DECLARATION:
    C : integer

ALGORITHM:

    { Switching the value of A & B }
    C ← A
    A ← B
    B ← C
```

```
PROGRAM Switching
(Switch the value of A & B )

DECLARATION:
    A, B, C : integer

ALGORITHM:

    { suppose that A is filled with 8 and B is filled with 5 }
    A ← 8
    B ← 5

    { print A and B values before switching }
    write("A and B values before switching: ")
    write("A = ", A)
    write("B = ", B)

    { Switch the value of A & B }
    C ← A
    A ← B
    B ← C

    { print A and B values after switching }
    write("A and B values after switching: ")
    write("A = ", A)
    write("B = ", B)
```

```
PROGRAM ModularSwitching
 (Switching the value of A & B)
DECLARATION:
      A, B : integer

      Procedure Filling(A, B: integer)
      ( Filling A & B with direct assignment )
      DECLARATION:
          { none }

      ALGORITHM:

          { suppose that A is filled with 8 and B is filled with 5 }
          A ← 8
          B ← 5

      Procedure Printing(A, B: integer)
      ( mencetak nilai A & B )
      DECLARATION:
          { none }

      ALGORITHM:

          write("A = ", A)
          write("B = ", B)

      Procedure Switching(A, B: integer)
      (Switching the value of A & B)
      DECLARATION:
          C : integer

      ALGORITHM:

          { Switching the value of A & B }
          C ← A
          A ← B
          B ← C

ALGORITHM:

    { suppose that A is filled with 8 and B is filled with }
    Filling(A, B)

    { print A and B values before switching }
    write("A and B values before switching: ")
    Printing(A, B)

    { Switching the value of A & B }
    Switching(A, B)

    { print A and B values after switching }
    write("A and B values after switching: ")
    Printing(A, B)
```

The Benefits of Modular

1. Avoid writing the same text repeatedly
2. Flexibility to make large programs (can be shared with many people, easy to read / understand)
3. Ease of tracking program errors (debug)

Two types of sub-program

1. *Procedure*
2. *Function*

Procedure :
do specific activities, receive input, but **does not** provide direct output

Function :
do specific activities, receive input, **provide direct output**

Procedure

Using Computer Devices in the Laboratory

Turning on Procedure:

1. Make sure your seating position and eye distance are ergonomic before operating the computer.
2. Make sure all power cables are connected to the mains.
3. Check whether there is an external device still connected. If there is, unplug it before the computer is turned on.
4. Turn on the CPU by pressing the power button or on the CPU cover.
5. turn on the monitor by pressing the power button if the monitor is still off.
6. wait for the boot process to finish until a desktop display appears on the screen.
7. computer ready to operate.

Defining a Procedure

1. The procedure name must be **unique**
2. The name of the procedure should start with **active verb** (e.g. widespreadCount, swap, maxSearch, enableMenu, etc.)
3. Procedure can have data **parameter** or not.
4. Data Parameter in the procedure header = Formal Parameters
5. Data Parameter is a communication medium between the procedure and the caller program.

Defining a Procedure

```
PROCEDURE ProcedureName(formal parameters declaration,if any)
{ procedure specification, contains an explanation of what is done
by this procedure }

DECLARATION:
{ all names used in the procedure and only apply locally in the
procedure are defined here }


ALGORITHM:
{ The procedure body, containing the sequence of instructions }
```

Procedure example

```
PROCEDURE CalculateTriangleArea()
{ calculate the area of a triangle with the formula L = (base x
height) / 2 }

DECLARATION:
      base   : real    { length of the triangular base, in cm}
      height : real    { height of triangle, in cm}
      area   : real    { area of triangle, in cm2}

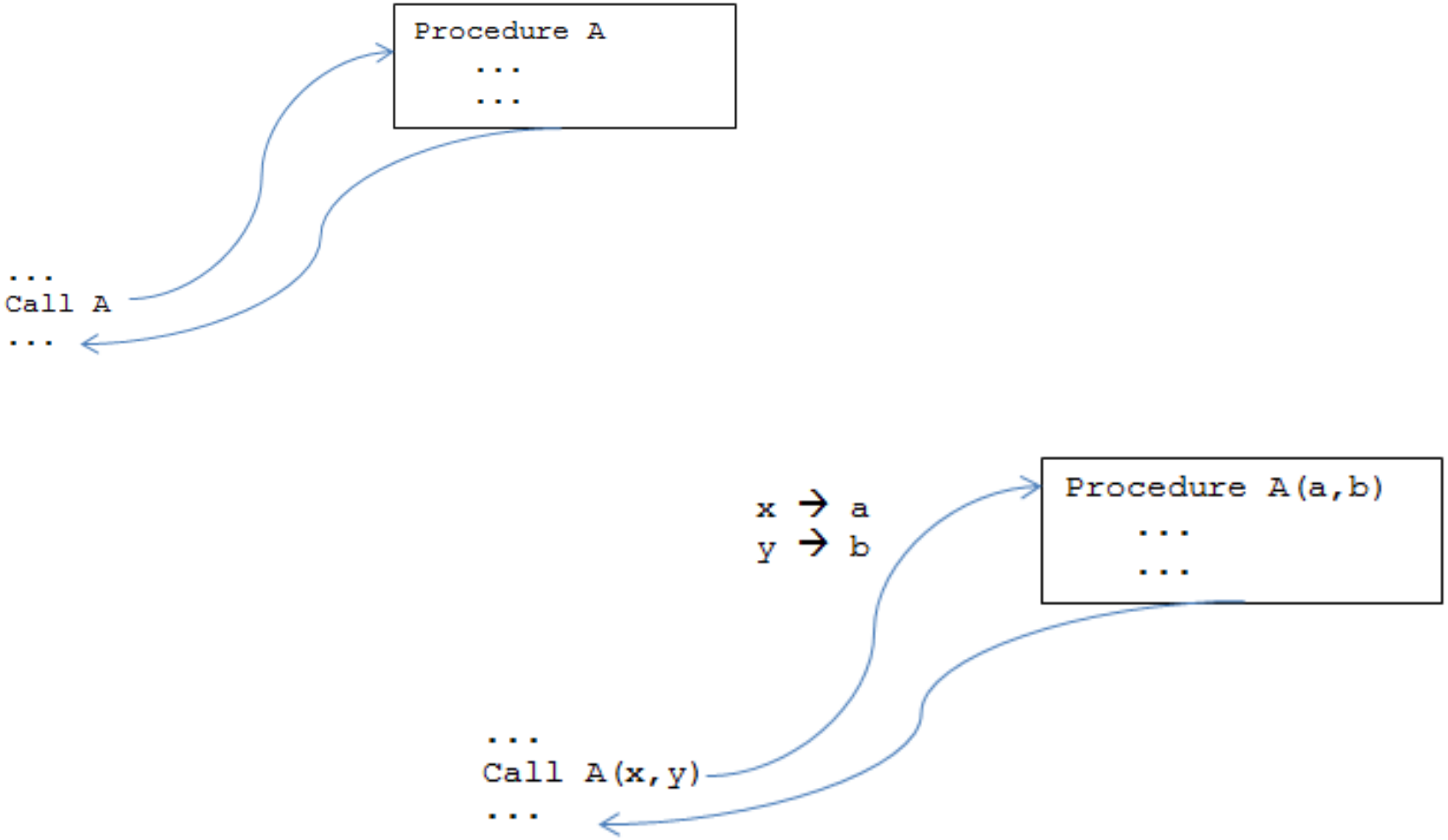

ALGORITHM:
    read(base, height)
    area ← (base * height)/2.0
    write(area)

    { area of the triangle is printed }
```

Procedure calling

1. The procedure is activated by calling it from the caller's program, along with its data parameter (if any)
2. The calling program can be a main program or other procedure (procedure A calls procedure B)
3. The data parameter included in the call is called the Actual Parameter
4. In order for the procedure to be recognized by the calling program, in the caller's program it is necessary to declare a prototype of the procedure.

Procedure calling



Procedure calling example

```
1   PROGRAM TriangleMainProgram
2   ( Calculate the area of N triangles. Example program calls the
3   Calculate Triangle Area procedure )
4
5   DECLARATION:
6       i : integer          { counting_variable }
7       N : integer          { numbers of triangles }
8
9       procedure CalculateTriangleArea()
10      { calculate the area of a triangle with the formula L = (base x
11      height) / 2 }
12
13  ALGORITHM:

14      write('Start calculating the area of N triangles)
15      read(N)                          { read the number of triangles }
16      for i ← 1 to N do
17        CalculateTriangleArea()              { call the procedure }
18      end
19      write('Finish')
```

Local Names vs Global Names

Local Name = constants, variables, types, and others (including formal parameters) declared in a procedure, only recognized in the body of the procedure.

Global Name = constants, variables, types, and others declared in the main program, recognized anywhere, in the main program or in the procedure

Formal Parameter vs Actual

Formal Parameter = The parameters declared in the procedure header.

Actual Parameter = The parameters included in the procedure call.

Rules to consider:

1. The actual number & order of parameters in the procedure call must be the same as the formal parameters.
2. Each actual parameter must be the same type as the formal parameter
3. Each actual parameter must be expressed in accordance with principle, according to the type of formal parameters

Formal Parameter types

1. *input parameter*
2. *output parameter*
3. *input/output parameter*

Input Parameter

1. The parameter whose value is used as input for the procedure
2. Often called as *parameter by value*
3. When the procedure is called, the actual parameter value is filled into the corresponding formal parameter.
4. Changing parameter values in the procedure body does not change the actual parameter values.
5. Input parameters are declared in the procedure header using the **input** keyword in front of it.

Input Parameter example

```
PROCEDURE CalculateTriangleArea(input base, height : real)
{ calculate the area of a triangle with the formula L = (base x
height) / 2 }

DECLARATION:
      area    : real    { area of triangle, in cm2}


ALGORITHM:
    area ← (base * height)/2.0
    write(area)

    { area of the triangle is printed }
```

```
1   PROGRAM TriangleMainProgram
2   ( Calculate the area of a triangles. Example program calling the
3   CalculateTriangleArea procedure )
4
5   DECLARATION:
6       b : real            { base of the triangle }
7       h : real            { height of the triangle }
8
9       procedure CalculateTriangleArea(input base, height : real)
10      { calculate the area of a triangle with the formula L = (base x
11      height) / 2 }
12
13  ALGORITHM:

14      read(b, h)                      { read base & height of triangles }
15      CalculateTriangleArea(b, h)     { call the procedure }
16      write('Finish')
```

Output Parameter

1. The parameter that holds the output value that the caller program will later use.
2. Also often called as *parameter by reference*
3. When a procedure is called, the actual parameter reference replaces the corresponding formal parameter.
4. Changing the parameter values in the procedure body will change the actual parameter values.
5. Output parameters are declared in the procedure header using the **output** keyword in front of it.

Output Parameter example

```
PROCEDURE CalculateTriangleArea(input base, height : real;
                                output area : real)
{ calculate the area of a triangle with the formula area = (base x
height) / 2 }

DECLARATION:
      { none }

ALGORITHM:
    area ← (base * height)/2.0
    { area of the triangle is calculated }
```

```
1   PROGRAM TriangleMainProgram
2   ( Calculate the area of a triangles. Example program calling the
3   CalculateTriangleArea procedure )
4
5   DECLARATION:
6       b : real            { base of the triangle }
7       h : real            { height of the triangle }
8       a : real            { area of the triangle }
9
10      procedure CalculateTriangleArea(input base, height : real;
11                                      output area : real)
12      { calculate the area of a triangle with the formula area = (base
13      x height) / 2 }
14
15  ALGORITHM:

16      read(b, h)                      { read base & height of triangles }
17      CalculateTriangleArea(b, h, a)      { call the procedure }
18      write(a)                        { printing the output area }
19      write('Finish')
```

Input/Output Parameter

1. Parameters that accommodate input / output that the caller program will later use.
2. When a procedure is called, the actual parameter reference replaces the corresponding formal parameter. In addition, the actual parameter values are also copied into the corresponding formal parameters
3. Changing the parameter values in the procedure body will change the actual parameter values.
4. Input/Output parameters are declared in the procedure header using the input/output keywords in front of them.

Input/Output Parameter example

```
PROCEDURE Switch(input/output A, B: integer)
{ switched the value of A & B }

DECLARATION:
      C : integer              { temporary variable }


ALGORITHM:
    { switching the value of A & B }
    C ← A
    A ← B
    B ← C
```

```
1   PROGRAM SwitchingMainProgram
2   (switching the value of A & B. Example program calling the Switch
3   procedure )
4
5   DECLARATION:
6       m, n : integer      { the switched variable }
7
8       Procedure Switch(input/output A, B: integer)
9               (switched the value of A & B)
10  ALGORITHM:
11      { initializing m with 8 and n with 5 }
12      m ← 8
13      n ← 5
14
15      write("before swithed :",m,n)
16      Switch(m,n)   { switched the values of m and n }
17      write("after swithed :",m,n)
```

Procedure translation

```
PROCEDURE CalculateTriangleArea(input base, height : real;
                                output area : real)
{ calculate the area of a triangle with the formula area = (base x
height) / 2 }

DECLARATION:
      { none }

ALGORITHM:
    area ← (base * height)/2.0
    { area of the triangle is calculated }
```

```
1   PROGRAM TriangleMainProgram
2   ( Calculate the area of a triangles. Example program calling the
3   CalculateTriangleArea procedure )
4
5   DECLARATION:
6       b : real            { base of the triangle }
7       h : real            { height of the triangle }
8       a : real            { area of the triangle }
9
10      procedure CalculateTriangleArea(input base, height : real;
11                                      output area : real)
12      { calculate the area of a triangle with the formula area = (base
13      x height) / 2 }
14
15  ALGORITHM:
16      read(b, h)                      { read base & height of triangles }
17      CalculateTriangleArea(b, h, a)      { call the procedure }
18      write(a)                        { printing the output area }
19      write('Finish')
```

Procedure translation into C program

```c
/* PROGRAM TriangleMainProgram */
/* Calculate the area of a triangles.
Example program calling the CalculateTriangleArea procedure .*/

#include <stdio.h>

/* DECLARATION: */
/* all names that are used globally declared here */

/* procedure and function prototypes declared here */
/* calculate the area of a triangle with the formula area = (base x height) / 2 */
void CalculateTriangleArea( float base, float height, float *area);

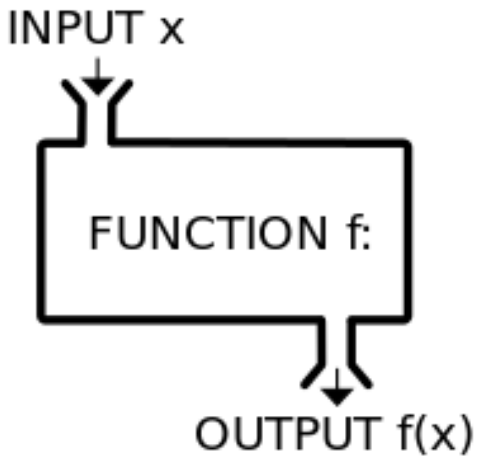
main()
{
    /* DECLARATION: */
    /* all names that are used locally declared here */
    float b;    /* base of the triangle  */
    float h;    /* height of the triangle */
    float a;    /* area of the triangle */

    /* ALGORITHM: */
    /* all program's instructions are written here */
    printf("Triangle base? "); scanf("%f", &b); printf("\nTriangle height? "); scanf("%f", &h);
    CalculateTriangleArea(b, h, &a);
    printf("Triangle Area = %.2f \n", a);
    printf("Finish");
}

void CalculateTriangleArea( float base, float height, float *area)
/* calculate the area of a triangle with the formula area = (base x height) / 2 */
{
    /* DECLARATION: */
       /* none */

    /* ALGORITHM */
    *area = (base * height)/2.0;
}
```

Function



The function accepts input and produces output.

Wikipedia

Definition

A function is a subprogram that receives data input and **returns** a value of a certain type (base type/composite type).

Function

1. The function name must be **unique**
2. The function must have data parameter and always be **input**. The function also always produces an **output** (not through explicit output parameters)
3. The value generated by the function can be either a basic type or composite.

Defining a Function

```
FUNCTION FunctionName(input parameters declaration) → output type
{ function specification, contains an explanation of what is done by
this function }

DECLARATION:
{ all names used in the function and only recognized locally in the
function are defined here }


ALGORITHM:
{ The function body, containing the sequence of instructions to
produce the output returned by the function }
```

Function Example

```
FUNCTION F(x : real) → real
{ returned the value of F(x) = 2x² + 5x - 8 }

DECLARATION:
{ none }


ALGORITHM:
      return 2.0 * x * x + 5.0 * x - 8.0
```

Function Example

```
FUNCTION IsLeapYear(year : integer) → boolean
{ returns true if the year is a leap year, false if not }

DECLARATION:
{ none }


ALGORITHM:
      if (year mod 4 = 0 and year mod 100 ≠ 0) or
         (year mod 400 = 0) then
           return true       {leap year}
      else
           return false      {non leap year}
      endif
```

Function calling

1. The function is activated by calling it from the calling program, along with the actual data parameter.
2. The calling program can be a main program or other function (function A calls function B)

Function calling

Because functions produce values, they can be treated in two ways:

1. The value returned by the function is stored in a **variable** of the **same type** as the function.

   Variable ← FunctionName(actual parameters)

   example:

   ```
   y ← F(5)
   m ← Maks(p,q)
   ```

2. The value returned by the function can be directly manipulated

   example:

   ```
   write(F(5))
   peak ← 10 * Maks(p,q)
   ```

example

```
FUNCTION Max(a, b : integer) → integer
{ returns the greatest value of a and b }

DECLARATION:
{ none }


ALGORITHM:
      if (a ≥ b) then
            return a
      else
            return b
      endif
```

```
1   PROGRAM MaxMainProgram
2   (The main program that calls the Max function to determine the
3   largest value of two input integers a and b)
4
5   DECLARATION:
6       firstInt,secondInt : integer        {values that are compared}
7       maxValue : integer                  {maximum value}
8
9       FUNCTION Max(a, b : integer) → integer
10      { returns the greatest value of a and b }

11  ALGORITHM:

12      read(firstInt, secondInt)           { read the values }
13      maxValue ← Max(firstInt, secondInt)     {call Max() function}
14      write(maxValue)
```

example

```
FUNCTION IsLeapYear(year : integer) → boolean
{ returns true if the year is a leap year, false if not }

DECLARATION:
{ none }


ALGORITHM:
      if (year mod 4 = 0 and year mod 100 ≠ 0) or
         (year mod 400 = 0) then
           return true       {leap year}
      else
           return false      {non leap year}
      endif
```

```
1   PROGRAM LeapYearMainProgram
2   (The main program determines whether a year is a leap year. Example
3   of IsLeapYear function call)
4
5   DECLARATION:
6       year : integer          {year that will be evaluated}
7
8       FUNCTION IsLeapYear(year : integer) → boolean
9       { returns true if the year is a leap year, false if not }
10  ALGORITHM:

11      read(year)                      { read year }
12      if IsLeapYear(year) then        { calling IsLeapYear()}
13        write("Leap Year")
14      else
15        write("Non Leap Year")
16      endif
```

Translation

```
FUNCTION Max(a, b : integer) → integer
{ returns the greatest value of a and b }

DECLARATION:
{ none }


ALGORITHM:
      if (a ≥ b) then
            return a
      else
            return b
      endif
```

```
1   PROGRAM MaxMainProgram
2   (The main program that calls the Max function to determine the
3   largest value of two input integers a and b)
4
5   DECLARATION:
6       firstInt,secondInt : integer       {values that are compared}
7       maxValue : integer                 {maximum value}
8
9       FUNCTION Max(a, b : integer) → integer
10      { returns the greatest value of a and b }
11  ALGORITHM:
12      read(firstInt, secondInt)          { read the values }
13      maxValue ← Max(firstInt, secondInt)     {call Max() function}
14      write(maxValue)
```

```c
/* PROGRAM MaxMainProgram */
/* The main program that calls the Max function
to determine the largest value of two input integers a and b) .*/

#include <stdio.h>

/* DECLARATION: */
/* all names that are used globally declared here */

/* procedure and function prototypes declared here */
/* returns the greatest value of a and b  */
int Max(int a, int b);

main()
{
    /* DECLARATION: */
    /* all names that are used locally declared here */
    /* DECLARATION */
    int firstInt,secondInt ;        /* value to be compared */
    int MaxValue;                   /* maximum value */

    /* ALGORITHM: */
    /* all program's instructions are written here */
    printf("firstInt = "); scanf("%d", &firstInt);
    printf("secondInt = "); scanf("%d", &secondInt);
    MaxValue = Max(firstInt, secondInt);
    printf("Maximum Value = %d \n", MaxValue);
}

int Max(int a, int b)
/* returns the largest value of a and b */
{
    /* DECLARATION */
        /* nothing */

    /* ALGORITHM */
    if (a >= b)
    {
        return a;
    }
    else
    {
        return b;
    }
}
```

Translation

```
FUNCTION IsLeapYear(year : integer) → boolean
{ returns true if the year is a leap year, false if not

DECLARATION:
{ none }


ALGORITHM:
      if (year mod 4 = 0 and year mod 100 ≠ 0) or
         (year mod 400 = 0) then
           return true      {leap year}
      else
           return false     {non leap year}
      endif
```

```
1   PROGRAM LeapYearMainProgram
2   (The main program determines whether a year is a leap year. Examp
3   of IsLeapYear function call)
4
5   DECLARATION:
6       year : integer          {year that will be evaluated}
7
8       FUNCTION IsLeapYear(year : integer) → boolean
9       { returns true if the year is a leap year, false if not }
10  ALGORITHM:
11      read(year)                      { read year }
12      if IsLeapYear(year) then        { calling IsLeapYear()}
13        write("Leap Year")
14      else
15        write("Non Leap Year")
16      endif
```

```c
/* PROGRAM LeapYearMainProgram */
/* The main program determines whether a year is a leap year.
Example of IsLeapYear function call .*/

#include <stdio.h>

/* DECLARATION: */
/* all names that are used globally declared here */
typedef enum {false=0, true=1} boolean; /* Boolean data type */

/* procedure and function prototypes declared here */
/* returns true if the year is a leap year, false if not */
boolean IsLeapYear(int year);

main()
{
    /* DECLARATION: */
    /* all names that are used locally declared here */
    /* DECLARATION */
    int year;

    /* ALGORITHM: */
    /* all program's instructions are written here */
    printf("year = "); scanf("%d", &year);      /* read year */
    if (IsLeapYear(year))           /* calling IsLeapYear() function*/
        printf("Leap Year\n");
    else
        printf("Non Leap Year\n");
}

boolean IsLeapYear(int year)
/* returns true if the year is a leap year, false if not */
{
    /* DECLARATION */
        /* nothing */

    /* ALGORITHM */
    if (((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0))
        return true;
    else
        return false;
}
```

Exercise

```
Procedure HitungRataRata(input N : integer, output u: real)
{ menghitung rata-rata N buah bilangan bulat yang dibaca dari papan
ketik. N adalah banyaknya data, nilainya sudah terdefinisi,
sedangkan u adalah luaran yang berisi rata-rata seluruh data.}

DEKLARASI
     x : real          { data bilangan bulat yang dibaca dari papan
                       ketik }
     i : integer       { pencacah pengulangan }
     jumlah : integer  { jumlah seluruh data }

ALGORITMA:
   jumlah ← 0   { inisialisasi }
   for i ← 1 to N do
     read(x)
     jumlah ← jumlah + x
   end for
   u ← jumlah/N

   { u berisi rata-rata seluruh data yang diinputkan }
```

```
PROGRAM RerataNilaiUjian
{ Program menghitung nilai rata-rata ujian N orang mahasiswa }


DEKLARASI
   N : integer         { jumlah peserta ujian }
   rerata : real       { nilai rata-rata seluruh peserta }

   Procedure HitungRataRata(input N : integer, output u: real)
   { menghitung rata-rata N buah bilangan bulat yang dibaca dari
   papan ketik. N adalah banyaknya data, nilainya sudah
   terdefinisi, sedangkan u adalah luaran yang berisi rata-rata
   seluruh data.}

ALGORITMA:
   read(N)                    { tentukan jumlah peserta ujian }
   HitungRataRata(N, rerata)     { panggil prosedur }
   write(rerata)                 { cetak nilai rata-rata }
   if rerata < 50 then
     write ('nilai rata-rata ujian tidak bagus')
   else
     write ('nilai rata-rata ujian bagus')
   end if
```

Exercise

```c
/* PROGRAM ReRataNilaiUjian
{ Program menghitung nilai rata-rata ujian N orang mahasiswa }
 */

#include <stdio.h>

/* Deklarasi purwarupa fungsi yang digunakan */
void HitungRataRata(int N, float *u);
/* { menghitung rata-rata N buah bilangan bulat yang dibaca dari papan ketik.
     N adalah banyaknya data, nilainya sudah terdefinisi,
     sedangkan u adalah luaran yang berisi rata-rata seluruh data.}
 */

main()
{
    /* DEKLARASI */
    int N;          /* jumlah peserta ujian */
    float rerata;   /*{ nilai rata-rata seluruh peserta }*/

    /* PROGRAM UTAMA */
    printf("Jumlah peserta ujian? = "); scanf("%d", &N);
    HitungRataRata(N, &rerata);     /*{ panggil prosedur }*/
    printf("Nilai rata-rata = %f \n", rerata);      /*{ cetak nilai rata-rata }*/
    if (rerata < 50)
    {
        printf("nilai rata-rata ujian tidak bagus \n");
    }
    else
    {
        printf("nilai rata-rata ujian bagus \n");
    }
}
```

```c

void HitungRataRata(int N, float *u)
/* { menghitung rata-rata N buah bilangan bulat yang dibaca dari papan ketik.
     N adalah banyaknya data, nilainya sudah terdefinisi,
     sedangkan u adalah luaran yang berisi rata-rata seluruh data.}
 */
{
    /* DEKLARASI */
    float x;    /*{ data bilangan bulat yang dibaca dari papan ketik }*/
    int i;      /*{ pencacah pengulangan }*/
    float jumlah;   /*{ jumlah seluruh data }*/


    /* ALGORITMA */
    jumlah = 0; /*{ inisialisasi }*/
    for (i = 1; i <= N; i++)
    {
        printf("Nilai ujian yang ke-%d = ", i); scanf("%f", &x);
        jumlah = jumlah + x;
    }
    *u = jumlah / N;
}
```

exercise

```
function TitikTengah(P1,P2 : Titik) → Titik
{ mengembalikan titik tengah dari P1 dan P2 }

DEKLARASI
      Pt : Titik { titik tengah }

ALGORITMA:
      Pt.x ← (P1.x + P2.x)/2
      Pt.y ← (P1.y + P2.y)/2
      return Pt
```

```
Titik P1 :
 x = 2
 y = 2
Titik P2 :
 x = 4
 y = 4
Titik tengah : (3.000000,3.000000)
```

```
PROGRAM MenghitungTitikTengah
{ Program utama untuk membaca dua buah titik, menghitung titik
tengahnya, dan mencetak titik tengah tersebut }


DEKLARASI
   type Titik : record
   < x : real,
     y : real
   >
   P1, P2 : Titik
   Pt : Titik { titik tengah }

   function TitikTengah(P1,P2 : Titik) → Titik
   { mengembalikan titik tengah dari P1 dan P2 }

ALGORITMA:
   read (P1.x, P1.y)        { baca titik P1 }
   read (P2.x, P2.y)        { baca titik P2 }
   Pt ← TitikTengah(P1, P2)
   Write (Pt.x, Pt.y)
```

exercise

```c
/* PROGRAM MenghitungTitikTengah
{ Program utama untuk membaca dua buah titik, menghitung titik tengahnya,
dan mencetak titik tengah tersebut } */

#include <stdio.h>

typedef struct {float x;
                float y;
                } Titik;

/* Deklarasi purwarupa fungsi yang digunakan */
Titik TitikTengah(Titik P1, Titik P2);
/*{ mengembalikan titik tengah dari P1 dan P2 } */

main()
{
    /* DEKLARASI */
    Titik P1, P2;
    Titik Pt;           /* Titik tengah */

    /* PROGRAM UTAMA */
    printf("Titik P1 : \n");
    printf(" x = "); scanf("%f", &P1.x);
    printf(" y = "); scanf("%f", &P1.y);

    printf("Titik P2 : \n");
    printf(" x = "); scanf("%f", &P2.x);
    printf(" y = "); scanf("%f", &P2.y);
    Pt = TitikTengah(P1, P2);
    printf("Titik tengah : (%f,%f) \n", Pt.x, Pt.y);
}
```

```c

Titik TitikTengah(Titik P1, Titik P2)
/*{ mengembalikan titik tengah dari P1 dan P2 } */
{
    /* DEKLARASI */
    Titik Pt;

    /* ALGORITMA */
    Pt.x = (P1.x + P2.x) / 2;
    Pt.y = (P1.y + P2.y) / 2;
    return Pt;
}
```

exercise

1. Buatlah fungsi **jarak** yang menerima masukan dua buah titik P1(x,y) dan P2(x,y) dan menghitung jarak kedua titik tersebut. Gunakan rumus Euclidian untuk menghitung jarak, d:

$$d = \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$$

2. Buatlah fungsi **apakah_i** yang memberikan nilai true jika karakter yang diterimanya adalah huruf 'i' dan false jika tidak.

3. Buatlah fungsi **roman** yang menerima angka dalam sistem desimal (1..10) dan mengembalikan angka romawinya.

Bibliography

- Munir, Lidya, “Algoritma & Pemrograman”, 6th Ed, 2016 Penerbit Informatika.
- Kernighan, Ritchie, “The C Programming Language”, 2nd Ed, Prentice Hall.